

The Kernel

When you type a command name, that program is run and a *process* is created for it

- Processes are user run programs
- The kernel is always run, it manages hardware

You can get info about the kernel using `uname`

uname

option

<code>-a</code>	all info
<code>-n</code>	system node name i.e. network hostname
<code>-s</code>	kernel name
<code>-v</code>	kernel version
<code>-r</code>	kernel version number
<code>-m</code>	machine info e.g. CPU code

```
james@laptop:~$ uname -a
Linux laptop 6.11.0-17-generic #17~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC Mon Jan 20 22:48:29 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

Examining Processes

ps [options]

process status

three different types of options:

<code>-</code> single dash	can be grouped together e.g. <code>-Ax</code>
<code>()</code> no dash	can be grouped together
<code>--</code> two dash	can't be grouped together

options

<code>-A -e</code>	all processes on the system
<code>x</code>	all processes owned by the user
<code>--h</code>	help
<code>v</code>	extra information

Output Column Names

```
james@laptop:~$ ps -A
  PID TTY          TIME CMD
    1 ?            00:00:04 systemd
    2 ?            00:00:00 kthreadd
```

- **PID** Process ID
- **PPID** Parent Process ID
- **TTY** TeleTYpe code to identify a terminal
- **TIME, %CPU** CPU time consumed/percentage when executed
- **CPU Priority** Default value 0 - positive values *reduce* priority, negative *increase*
- **Command** launch process

- **RSS, %MEM** memory - Resident Set Size (program + data)

example

view a particular process

```
ps -A | grep bash
```

example

view parent/child processes

```
ps -A --forest
```

```

9612 ?      00:02:35  \_ code
9614 ?      00:00:00  |  \_ code
9615 ?      00:00:00  |  \_ code
9677 ?      00:14:45  |  |  \_ code
20329 ?     00:00:08  |  |  \_ code

```

Foreground & background processes

Foreground process - takes over control of the terminal, preventing you from doing other work

- **Ctrl+Z** pauses the program and gives you back the terminal
- **fg** restores the foreground program
- **bg** restore the foreground program, but run it in the background

Resource usage

top

examining process **CPU usage**

updates every few seconds

option

-d delay	specify delay between updates
-p pid	to monitor a specific process

```

top - 11:30:19 up 5:37, 1 user, load average: 0.14, 0.29, 0.28
Tasks: 365 total, 1 running, 364 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.9 us, 1.1 sy, 0.0 ni, 96.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 15745.7 total, 5070.7 free, 6255.2 used, 6618.8 buff/cache
MiB Swap: 16384.0 total, 16384.0 free, 0.0 used. 9490.5 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 3275 james    20   0   25.6g 240188 163872 S   12.0   1.5   22:25.30 Xorg
 3667 james    20   0 4991112 353884 145604 S    7.3   2.2   17:16.96 gnome-shell
 9530 james    20   0 1183312 230192  74436 S    3.3   1.4    1:00.71 evince
20784 james    20   0 1356024 129916  77908 S    3.0   0.8    2:15.01 jamesdsp
20952 james    20   0 4869324 404244 197968 S    3.0   2.5    2:01.81 spotify
 3162 james     9  -11  143016  34724  11176 S    1.3   0.2    0:37.91 pipewire-pulse

```

load average (top line): shows *current load average* : two previous measures

0 = no programs demanding cpu

1 = one program running CPU-intensive tasks

higher averages = multiple programs competing for CPU time

when it's running you can use these commands:

k kill a process - will ask for PID

q quit

free

for examining Memory usage# Process priorities

nice launch a program with a specified priority

renice change a running process priority

```
nice [argument] [command]
```

You can assign a priority in 3 ways:

- `nice -12 number-crunch data.txt` (*assigns 12 not -12*)
- `nice -n 12 number-crunch data.txt`
- `nice --adjustment=12 number-crunch data.txt`
run *number-crunch* at priority 12 and pass in *data.txt*

- 0 default
- -20 (highest)
- 19 (lowest)

```
renice priority [-p [pids]] [-g [pgrps]] [-u [users]]
```

specify:

- **priority**
- **pids** 1+ PIDs
- or **pgrps** 1+ group IDs
- or 1+ **users** usernames

example

```
renice 7 16580 -u pdavison tbaker
```

7 set *priority*

16580 to process with *PID*

pdavison tbaker and all processes owned by *users*

Killing processes

```
kill -s signal pid
```

sends a signal to a process

kills processes owned by the user who runs kill, root can kill any users processes

signal

- Linux has lots of signals (method to communicate with a process)
- each with a specific name
- You can specify a signature with its number (eg 9) or name (SIGKILL)

get all signals `kill -l`

```
james@laptop:~$ kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL    10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```

- **9)SIGKILL** kill without performing shutdown tasks
- **15)SIGTERM** *graceful shutdown* - kill but allows it to close open files, save data etc
- **15)SIGTERM** is default

nohup

When you log out of a session, the programs you started are terminated

To run a program that will keep running when you log out

e.g. run servers

```
nohup [program] [options]
```